

Experiment 12: Minimax Algorithm for Gaming

Aim

To implement the Minimax Algorithm using Python.

Objective

To find the best possible move for a player in a game.

Algorithm

1. Generate all possible moves.
 2. Assign scores to each move.
 3. Maximize the player's score.
 4. Minimize the opponent's score.
 5. Select the best move.
 6. Display the result.
-

Flowchart

Start

|

Generate Moves

|

Evaluate Scores

|

Find Best Move

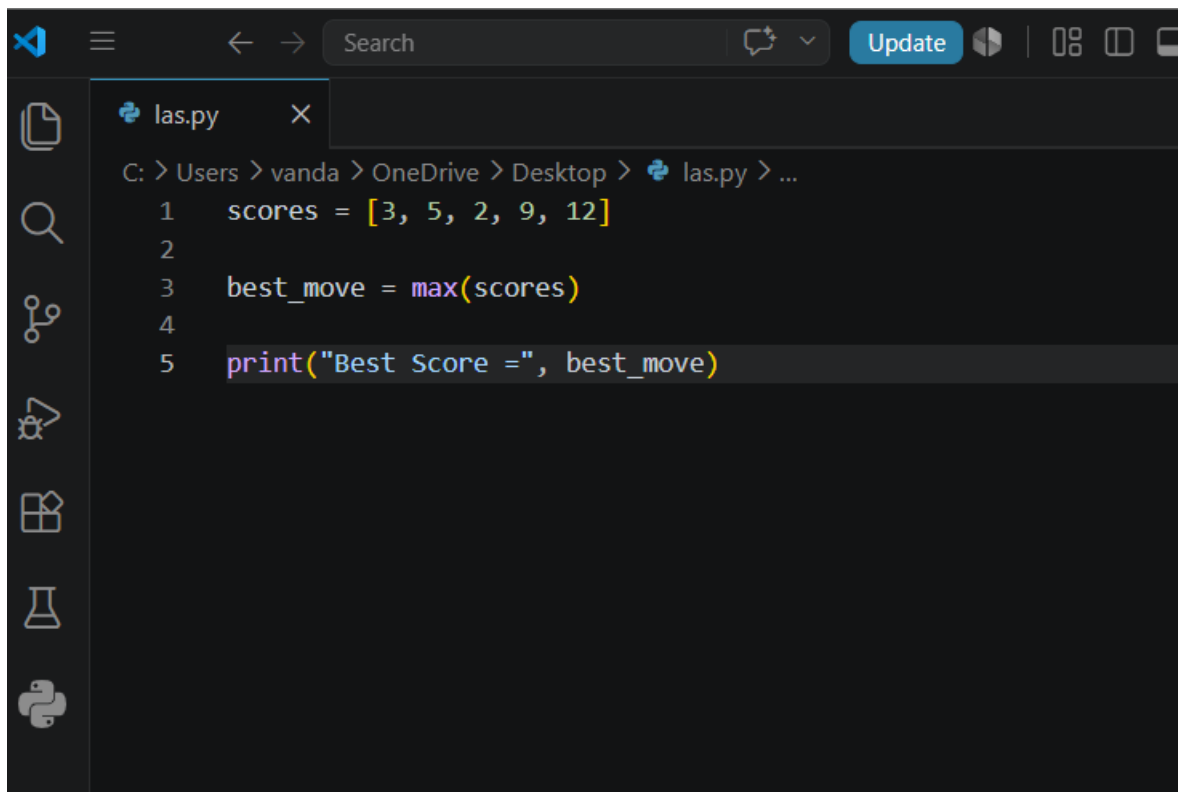
|

Display Result

|

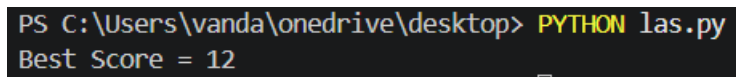
Stop

Code

A screenshot of a code editor interface. The top bar shows a search bar, a 'Search' button, and an 'Update' button. The left sidebar contains icons for file explorer, search, source control, and other tools. The main editor area shows a file named 'las.py' with the following code:

```
C: > Users > vanda > OneDrive > Desktop > las.py > ...  
1  scores = [3, 5, 2, 9, 12]  
2  
3  best_move = max(scores)  
4  
5  print("Best Score =", best_move)
```

Output:

A screenshot of a Windows command prompt. The prompt shows the command 'PYTHON las.py' being executed, and the output 'Best Score = 12' is displayed.

```
PS C:\Users\vanda\onedrive\desktop> PYTHON las.py  
Best Score = 12
```

Result

The best move was selected successfully using the Minimax algorithm.

Conclusion

The Minimax algorithm helps in selecting the optimal move by evaluating all possible game states.